ARTIFICIAL INTELLIGENCE
LABORATORY PRACTICAL

IMPLEMENT GOAL STACK PLANNING FOR SOLVING THE BLOCKS WORLD PROBLEM

AND ACHIEVE A SPECIFIED GOAL CONFIGURATION

Name of Student	_____
Roll Number	_____
Class / Division	_____
Subject	Artificial Intelligence Laboratory
Academic Year	2026–2027
Date of Performance	_____
Date of Submission	_____

Department of Computer Engineering

1. Aim

To implement Goal Stack Planning for solving the Blocks World problem and to achieve a specified goal configuration of blocks using Python.

2. Objectives

- To understand the concept of planning in Artificial Intelligence.
- To represent the Blocks World problem using states, predicates, operators, initial state and goal state.
- To understand the purpose of a goal stack and how subgoals are handled.
- To generate a valid sequence of actions that transforms an initial arrangement into the required goal arrangement.
- To execute the Python program and verify the final configuration.

3. Problem Statement

Consider a set of blocks placed on a table or stacked on one another. A robot arm can hold only one block at a time. The task is to rearrange the blocks from a given initial configuration into a specified goal configuration by applying valid actions such as PICKUP, PUTDOWN, STACK and UNSTACK. The planner must produce an ordered sequence of actions that reaches the goal without violating the Blocks World constraints.

4. Theory

Planning in Artificial Intelligence is the process of finding a sequence of actions that changes the world from an initial state to a desired goal state. A planning system uses a representation of the current state, a set of actions with preconditions and effects, and a search or planning strategy to determine the required actions.

Goal Stack Planning is a classical planning technique in which goals and subgoals are placed on a stack. The planner works backward from the goal, selects an action that can achieve a goal, and places that action's preconditions on the stack. Once the preconditions are satisfied, the action can be executed. The process continues until all goals are achieved.

The Blocks World is a simplified planning environment containing blocks, a table and a robot hand. Each block can be on the table or on exactly one other block. A block can support at most one block, and only a clear block (one with nothing on top) can be picked up or used as the destination for stacking.

5. Blocks World Representation

Common predicates:

Predicate	Meaning
ON(X, Y)	Block X is placed on block Y.
ONTABLE(X)	Block X is directly on the table.
CLEAR(X)	There is no block on top of block X.
HOLDING(X)	The robot arm is holding block X.
ARMEMPTY	The robot arm is not holding any block.

6. Operators and Their Conditions

The four basic operators are:

Operator	Preconditions	Main effects
PICKUP(X)	ONTABLE(X), CLEAR(X), ARMEMPTY	HOLDING(X); remove ONTABLE(X), CLEAR(X), ARMEMPTY
PUTDOWN(X)	HOLDING(X)	ONTABLE(X), CLEAR(X), ARMEMPTY; remove HOLDING(X)
UNSTACK(X,Y)	ON(X,Y), CLEAR(X), ARMEMPTY	HOLDING(X), CLEAR(Y); remove ON(X,Y), CLEAR(X), ARMEMPTY
STACK(X,Y)	HOLDING(X), CLEAR(Y)	ON(X,Y), CLEAR(X), ARMEMPTY; remove HOLDING(X), CLEAR(Y)

7. Prerequisites

- Basic knowledge of Python programming.
- Understanding of lists, dictionaries, functions and classes.
- Basic understanding of states, goals and actions in Artificial Intelligence.

8. Software and Hardware Requirements

- Software: Python 3.x, VS Code / PyCharm / IDLE, and a terminal or command prompt.
- Libraries: Python standard library only; no external package is required.
- Hardware: Computer or laptop.

9. Algorithm / Procedure

1. Start the program.
2. Read the list of blocks and the initial arrangement.
3. Read the required goal tower from bottom to top.
4. Represent the current arrangement as a mapping: each block is either on the table or on another block.
5. For each block that must be placed on a support block, first clear the block to be moved and clear the destination support.
6. If the block is on the table, apply PICKUP; otherwise apply UNSTACK from its current support.
7. Apply STACK to place the held block on the required support.
8. Update the current state after every action and record the action in the plan.
9. After all goal relations are processed, display the action sequence and final configuration.
10. Stop.

10. Flowchart

START → Read blocks, initial state and goal tower → Initialize state and plan → Is the next goal relation already satisfied? → If yes, continue to the next relation → If no, clear the moving block and destination → PICKUP / UNSTACK → STACK on destination → Update state and record action → Are all goal relations satisfied? → If yes, display plan and final state → STOP.

Note: The flow is shown textually so it can be copied into a handwritten practical journal as a flowchart. Draw the decision steps as diamonds and action steps as rectangles if a graphical flowchart is required.

11. Python Program

Save the following program as **GoalStackPlanning.py**. This implementation uses a goal stack / subgoal-driven procedure for the common Blocks World case where the requested goal is one tower. The goal tower is supplied from bottom to top.

```
class BlocksWorld:
    def __init__(self, initial_state):
        # Example: {"A": "TABLE", "B": "TABLE", "C": "A"}
        # A value of "TABLE" means that the block is on the table.
        self.state = dict(initial_state)
        self.plan = []

    def block_above(self, block):
        """Return the block directly above 'block', if one exists."""
        for candidate, support in self.state.items():
            if support == block:
                return candidate
        return None

    def is_clear(self, block):
        return self.block_above(block) is None

    def move_to_table(self, block):
        """Move a clear block to the table using valid arm actions."""
        if not self.is_clear(block):
            raise ValueError(f"Cannot move {block}: it is not clear.")

        old_support = self.state[block]

        if old_support == "TABLE":
            return

        self.plan.append(f"UNSTACK({block}, {old_support})")
        self.state[block] = "IN_HAND"

        self.plan.append(f"PUTDOWN({block})")
        self.state[block] = "TABLE"

    def clear_block(self, block):
        """Clear a block by moving the block directly above it to the table."""
        while not self.is_clear(block):
            top_block = self.block_above(block)
            if top_block is None:
                break
            self.move_to_table(top_block)

    def move_onto(self, block, destination):
        """Place a clear block onto a clear destination block."""
        if block == destination:
            raise ValueError("A block cannot be placed on itself.")

        self.clear_block(block)
        self.clear_block(destination)

        if self.state[block] == destination:
            return

        old_support = self.state[block]

        if old_support == "TABLE":
            self.plan.append(f"PICKUP({block})")
        else:
            self.plan.append(f"UNSTACK({block}, {old_support})")

        self.state[block] = "IN_HAND"

        self.plan.append(f"STACK({block}, {destination})")
        self.state[block] = destination

    def solve_tower_goal(self, goal_tower):
        """
        goal_tower is ordered from bottom to top.
        Example: ["A", "B", "C"] means A on table, B on A, C on B.
        """
        if len(set(goal_tower)) != len(goal_tower):
            raise ValueError("A block appears more than once in the goal.")
```

```
        if not set(goal_tower).issubset(self.state.keys()):
            raise ValueError("Goal contains a block not present in the initial state.")

        # Ensure the base block is on the table.
        base = goal_tower[0]
        self.clear_block(base)
        if self.state[base] != "TABLE":
            self.move_to_table(base)

        # Build the tower from bottom to top.
        for index in range(1, len(goal_tower)):
            block = goal_tower[index]
            support = goal_tower[index - 1]
            self.move_onto(block, support)

        return self.plan


def print_configuration(state):
    print("Current configuration (block: support):")
    for block in sorted(state):
        print(f"  {block}: {state[block]}")


if __name__ == "__main__":
    # Initial state:
    # A is on the table, B is on the table, and C is on A.
    initial = {
        "A": "TABLE",
        "B": "TABLE",
        "C": "A"
    }

    # Goal tower from bottom to top:
    # A on table, B on A, and C on B.
    goal = ["A", "B", "C"]

    planner = BlocksWorld(initial)

    print("INITIAL CONFIGURATION")
    print_configuration(planner.state)

    print("\\nGOAL TOWER (bottom to top):", " -> ".join(goal))

    actions = planner.solve_tower_goal(goal)

    print("\\nGENERATED PLAN")
    if actions:
        for number, action in enumerate(actions, start=1):
            print(f"{number}. {action}")
    else:
        print("No actions required; the goal is already satisfied.")

    print("\\nFINAL CONFIGURATION")
    print_configuration(planner.state)

    print("\\nGOAL ACHIEVED:",
          planner.state["A"] == "TABLE"
          and planner.state["B"] == "A"
          and planner.state["C"] == "B")
```

12. Explanation of the Program

- **state:** A Python dictionary stores each block and the block or table supporting it.
- **block_above():** Finds the block directly above a specified block.
- **is_clear():** Checks whether a block has no block above it.
- **move_to_table():** Moves a clear block from another block to the table using UNSTACK and PUTDOWN.
- **clear_block():** Repeatedly moves blocks above a target block to the table until the target is clear.
- **move_onto():** Clears the moving block and destination, then uses PICKUP or UNSTACK followed by STACK.
- **solve_tower_goal():** Processes the requested tower from bottom to top and records the actions.

The code is intentionally scoped to a single-tower goal configuration. It demonstrates the goal/subgoal ordering used in a goal-stack-style Blocks World practical; it is not a general-purpose STRIPS planner for every possible arrangement of multiple separate towers.

13. Sample Input

Initial state:

```
A: TABLE\nB: TABLE\nC: A
```

Goal tower (bottom to top):

```
A -> B -> C
```

This means A must remain on the table, B must be placed on A, and C must be placed on B.

14. Expected Output

```
INITIAL CONFIGURATION\nCurrent configuration (block: support):\n  A: TABLE\n  B: TABLE\n  C: A\n\nGOAL TOWER (bottom to top)
```

The output above is the expected result for the sample input. Run the program on your computer and write the actual output observed in your journal.

15. Test Cases

Test	Initial configuration	Goal	Expected result
1	A and B on table; C on A	A → B → C	Planner moves C aside, places B on A, then C on B.
2	A → B → C already forms the required tower	A → B → C	No actions required; goal is already satisfied.
3	A, B and C all on table	A → B → C	Planner places B on A and then C on B.

16. Applications

- Robotic arm manipulation and object rearrangement.
- Automated warehouse and material-handling systems.
- Game AI and puzzle-solving systems.
- Task planning in intelligent agents.
- Manufacturing and assembly planning.

17. Advantages

- Breaks a complex goal into smaller subgoals.
- Provides an understandable sequence of actions.
- Uses explicit representations of states and operators.
- Useful for demonstrating classical AI planning concepts.

18. Limitations

- The presented program is designed for a single tower goal, not arbitrary multi-tower goals.
- It assumes one block can be directly above another block at a time.
- It does not optimize the plan for the minimum number of actions.

- Real robotic systems also need sensing, motion control and collision avoidance.

19. Result

The Blocks World problem was represented using block-support states and basic operators. A Python program was implemented to generate a valid sequence of actions for the specified single-tower goal configuration. For the sample case, the final arrangement is A on the table, B on A and C on B, and the goal condition is satisfied.

20. Conclusion

Goal Stack Planning demonstrates how an AI planning problem can be decomposed into goals and subgoals. By representing the Blocks World using states and operators, the program determines the actions needed to transform an initial arrangement into a desired tower. This practical provides an introduction to symbolic representation, preconditions, effects and action sequencing in Artificial Intelligence.

21. Viva Questions and Answers

1. What is planning in Artificial Intelligence?

Planning is the process of finding an ordered sequence of actions that transforms an initial state into a desired goal state.

2. What is Goal Stack Planning?

It is a classical planning method that stores goals and subgoals on a stack and resolves them by selecting actions and satisfying their preconditions.

3. What is the Blocks World problem?

It is a planning problem in which blocks are rearranged on a table using a robot arm.

4. What is an initial state?

It is the description of the world before any planning action is performed.

5. What is a goal state?

It is the desired arrangement or condition that the planner must achieve.

6. What is a predicate?

A predicate is a logical expression used to describe a condition in a state, such as ON(A, B).

7. What does CLEAR(X) mean?

It means that there is no block placed on top of block X.

8. What does ARMEMPTY mean?

It means that the robot arm is not holding any block.

9. What is the purpose of PICKUP?

PICKUP removes a clear block from the table and places it in the robot arm.

10. What is the purpose of PUTDOWN?

PUTDOWN places the block held by the robot arm onto the table.

11. What is the purpose of STACK?

STACK places the block held by the arm onto a clear destination block.

12. What is the purpose of UNSTACK?

UNSTACK removes a clear block from the block supporting it and places it in the robot arm.

13. What are preconditions?

Preconditions are the conditions that must be true before an action can be performed.

14. What are effects?

Effects are the changes to the world state caused by executing an action.

15. What is a subgoal?

A subgoal is a smaller condition that must be achieved as part of reaching the overall goal.

22. Short Oral Explanation

“In this practical, I implemented a goal-stack-style planning procedure for the Blocks World problem. I represented the arrangement using a dictionary, where each block stores its supporting block or TABLE. The planner checks which blocks are clear, moves obstructing blocks to the table, and then builds the required tower from bottom to top using PICKUP, UNSTACK, PUTDOWN and STACK actions. Finally, it displays the generated plan and verifies whether the goal configuration has been achieved.”

23. Lab Assignment / Further Work

- Modify the program to accept the initial state and goal tower from the user.
- Add validation to reject impossible or cyclic configurations.
- Extend the planner to support goals containing multiple separate towers.
- Compare the generated plan with a breadth-first search planner and measure the number of actions.

24. CO / PO Mapping (Suggested)

Use the exact CO and PO codes specified by your department or course instructor. The following is a general mapping and should be adjusted to match the official course outcome table.

Course Outcome (CO)	Possible contribution
CO: Apply AI problem-solving techniques	Represent a planning problem and generate a sequence of actions.
PO: Engineering knowledge	Apply programming and AI concepts to a problem.
PO: Problem analysis	Identify states, goals, operators and constraints.
PO: Design / development	Develop and test a planning program.

25. Assessment / Record

Assessment item	Record
Program execution	Successful / Needs correction: _____
Test cases performed	_____
Observed output	_____
Faculty remarks	_____
Faculty signature	_____